Question Bank — Week 5

Linked lists (Lecture 05) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **8**

Part C — Trace the code **8**

Part D — List state **9**

Part E — Complexity analysis **10**

Part F — Find and fix the bug **10**

Part G — Write the code **11**

> ***Answers are in a separate file:*** *`week05-answers.md`. Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. `Node(value, next=None)` and `LinkedList` are the classes of `dsa/linked_list.py`: a singly linked list with `head` and `_size`, and no tail pointer unless a question says so.*

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W5-M01 – W5-M40
B	Short answer and essay	W5-E1 – W5-E9
C	Trace the code	W5-T1 – W5-T5
D	List state — draw every step	W5-S1 – W5-S3
E	Complexity analysis	W5-K1 – W5-K3
F	Find and fix the bug	W5-B1 – W5-B4
G	Write the code — checked by `pytest`	W5-C1 – W5-C5

Part A — Multiple choice

W5-M01 [what] A node of a singly linked list holds:

- **a)** a value and a reference to the next node

- **b)** a value and its index in the list
- **c)** a value and references to every later node
- **d)** only a value; the list stores the order separately

W5-M02 [what] Reading `ll[i]` on a linked list of n nodes costs:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n)$
- **d)** amortised $O(1)$

W5-M03 [what] `push_front` costs:

- **a)** $O(n)$
- **b)** $O(1)$
- **c)** amortised $O(1)$
- **d)** $O(\log n)$

W5-M04 [what] append on a singly linked list **without** a tail reference costs:

- **a)** $O(1)$
- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$

W5-M05 [why] Which operation is $O(1)$ on a linked list but $O(n)$ on a dynamic array?

- **a)** inserting at the front
- **b)** reading element i
- **c)** appending at the end
- **d)** searching for a value

W5-M06 [why] Which operation is $O(1)$ on a dynamic array but $O(n)$ on a linked list?

- **a)** inserting at the front
- **b)** removing the first element
- **c)** reading element i
- **d)** `len`

W5-M07 [why] In `push_front`, why must `new.next = self.head` come **before** `self.head = new`?

- **a)** Python evaluates assignments right to left
- **b)** Done the other way round, `new.next` would point to `new` itself and the rest of the list would be lost
- **c)** It is only a style convention
- **d)** Otherwise `_size` would be wrong

W5-M08 [why] To remove a node from a singly linked list, you need a reference to:

- **a)** the head only
- **b)** the node after it
- **c)** the last node

- **d)** the node before it

W5-M09 [what] Which node is the special case in `remove(value)` without a sentinel?

- **a)** the head
- **b)** the last node
- **c)** the middle node
- **d)** none — there is no special case

W5-M10 [how] What is the running time of this loop over a linked list of n nodes?

```
for i in range(len(ll)):
    total += ll[i]
```

- **a)** $O(n)$
- **b)** $O(n \log n)$
- **c)** $O(n^2)$
- **d)** $O(1)$

W5-M11 [why] Why is `len(ll)` $O(1)$ in `dsa/linked_list.py`?

- **a)** Python caches the length of every object
- **b)** The list keeps `_size` up to date in every method that adds or removes
- **c)** Linked lists have a fixed size
- **d)** It is not — it walks the list

W5-M12 [how] Reversing a linked list in place needs how much extra space?

- **a)** $O(n)$
- **b)** $O(\log n)$
- **c)** $O(n^2)$
- **d)** $O(1)$

W5-M13 [what] On 64-bit CPython, a `Node` with `__slots__` takes about 48 bytes. One slot of an array of references takes:

- **a)** 8 bytes
- **b)** 48 bytes
- **c)** 1 byte
- **d)** it depends on the value stored

W5-M14 [why] A **doubly** linked list can do in $O(1)$ what a singly linked one cannot:

- **a)** read element i
- **b)** remove a node you already hold a reference to
- **c)** search for a value
- **d)** sort the list

W5-M15 [why] What is a sentinel (dummy) node for?

- **a)** to mark the end of the list instead of `None`
- **b)** to store the length
- **c)** to give every real node a predecessor, removing the special case for the head

- **d)** to make indexing $O(1)$

W5-M16 [why] A singly linked list **with** a tail reference. Removing the **last** node costs:

- **a)** $O(1)$ — the tail points at it
- **b)** amortised $O(1)$
- **c)** $O(\log n)$
- **d)** $O(n)$ — the new last node must be found from the head

W5-M17 [what] Python's `collections.deque` is implemented as:

- **a)** a doubly linked list of fixed-size blocks
- **b)** a dynamic array
- **c)** a hash table
- **d)** a singly linked list of single values

W5-M18 [why] Why does the traversal loop test `while node is not None:` rather than `while node.next is not None:`?

- **a)** It is faster
- **b)** The second version stops **on** the last node, so it never visits it — and crashes on an empty list
- **c)** `node.next` is not allowed in a condition
- **d)** There is no difference

W5-M19 [how] After `a = Node(1)`, `b = Node(2, a)`, `c = Node(3, b)`, what is `c.next.next.value`?

- **a)** 3
- **b)** 2
- **c)** 1
- **d)** it raises `AttributeError`

W5-M20 [what] In Python, what happens to a node that nothing refers to any more?

- **a)** it stays in memory until the program ends
- **b)** you must call `del` on it
- **c)** it becomes the new head
- **d)** the garbage collector reclaims it

W5-M21 [how] To insert a new node at index i ≥ 1, you walk to the node at index:

- **a)** i – 1
- **b)** i
- **c)** i + 1
- **d)** 0 only

W5-M22 [what] Where in this course do short linked lists appear inside another structure?

- **a)** in the dynamic array
- **b)** as the chains of a hash table
- **c)** in the heap
- **d)** in the course `Array`

W5-M23 [why] What is printed?

```
ll = LinkedList([1, 2, 3, 4])
node = ll.head.next
node.next = node.next.next
print(list(ll), len(ll))
```

- **a)** `[1, 2, 4] 3`
- **b)** `[1, 3, 4] 3`
- **c)** `[1, 2, 4] 4`
- **d)** `[1, 2, 3, 4] 4`

W5-M24 [how] On a list of 6 nodes, `insert_at(4, x)` runs. How many times does `prev = prev.next` run?

- **a)** 6
- **b)** 5
- **c)** 4
- **d)** 3

W5-M25 [what] `ll = LinkedList([1, 2, 3])`. Which call raises `IndexError`?

- **a)** `ll[3]`
- **b)** `ll.insert_at(3, 9)`
- **c)** `ll[-1]`
- **d)** `ll.insert_at(0, 9)`

W5-M26 [what] In a list with a sentinel node, the list is empty when:

- **a)** `self.sentinel is None`
- **b)** `self.sentinel.next is None`
- **c)** `self.sentinel.value is None`
- **d)** the sentinel has been removed

W5-M27 [how] In a doubly linked list, x is a node that is neither the first nor the last. Which two assignments remove it from the list?

- **a)** `x.next = x.prev` and `x.prev = x.next`
- **b)** `x.prev = None` and `x.next = None`
- **c)** `x.prev.next = x.next` and `x.next.prev = x.prev`
- **d)** `x.prev.next = x.prev` and `x.next.prev = x.next`

W5-M28 [how] A new node is inserted between two neighbouring nodes a and b of a **doubly** linked list. How many references must be set, counting the new node's own?

- **a)** 1
- **b)** 2
- **c)** 3
- **d)** 4

W5-M29 [why] A student adds `_tail` to the list. Their `append` decides "empty" by testing `self._tail is None`, and their `pop_front` updates `head` and `_size` but never `_tail`. What

does this print?

```
ll = LinkedList([7])
ll.pop_front()
ll.append(5)
print(list(ll), len(ll))
```

- **a)** `[] 1`
- **b)** `[5] 1`
- **c)** `[7, 5] 2`
- **d)** it raises `AttributeError`

W5-M30 [how] `reverse()` runs on 1 → 2 → 3. After the **first** pass of the loop body, what is the state?

- **a)** `prev` is `None`, `current` is `Node(2)`, `Node(1).next` is `Node(2)`
- **b)** `prev` is `Node(1)`, `current` is `Node(2)`, `Node(1).next` is `None`
- **c)** `prev` is `Node(2)`, `current` is `Node(3)`, `Node(2).next` is `Node(1)`
- **d)** `prev` is `Node(1)`, `current` is `Node(2)`, `Node(1).next` is `Node(2)`

W5-M31 [why] Suppose `insert_at` had **no** `index == 0` case and went straight to the walk. What would `insert_at(0, 9)` do to `[1, 2, 3]`?

- **a)** `[9, 1, 2, 3]`, as it should
- **b)** raise `IndexError`
- **c)** give `[1, 9, 2, 3]`: `range(-1)` is empty, so the node goes after the head
- **d)** loop for ever

W5-M32 [what] `ll = LinkedList([4, 2, 4])`, then `ll.remove(4)`. Now `list(ll)` is:

- **a)** `[2]`
- **b)** `[4, 2]`
- **c)** `[4, 2, 4]`
- **d)** `[2, 4]`

W5-M33 [why] Walking a dynamic array and walking a linked list are both $O(n)$, yet the array walk is usually several times faster. Why?

- **a)** The array walk is really $O(\log n)$
- **b)** Each step of the list walk must update `_size`
- **c)** The array's references are contiguous and the processor reads memory in blocks; nodes are scattered, so each `next` may jump elsewhere
- **d)** Python copies every node as the walk reaches it

W5-M34 [why] In a **circular** list the last node points back to the first. What does the usual loop `while node is not None:` do on it?

- **a)** It loops for ever, because no `next` is ever `None`
- **b)** It visits every node once and stops
- **c)** It raises `AttributeError` at the end
- **d)** It stops when it returns to the head

W5-M35 [how] What is printed?

```
a, b, c = Node(1), Node(2), Node(3)
a.next = b
b.next = c
b.next, c.next = None, b
print(a.next.value, c.next.value, b.next)
```

- **a)** `2 3 None`
- **b)** `2 2 None`
- **c)** `3 2 None`
- **d)** `2 None 3`

W5-M36 [what] `LinkedList().pop_front()`:

- **a)** returns `None`
- **b)** returns 0
- **c)** raises `AttributeError`
- **d)** raises `IndexError`

W5-M37 [why] You hold a reference `node` to a node in the middle of a long **singly** linked list. Inserting a new node immediately **before** it costs:

- **a)** $O(1)$, like inserting after it
- **b)** amortised $O(1)$
- **c)** $O(n)$: the node before it must be found by walking from the head
- **d)** $O(\log n)$

W5-M38 [how] What is printed?

```
ll = LinkedList()
for v in [3, 1, 2]:
    ll.push_front(v)
ll.append(ll.pop_front())
print(list(ll))
```

- **a)** `[1, 3, 2]`
- **b)** `[2, 1, 3]`
- **c)** `[3, 1, 2]`
- **d)** `[1, 2, 3]`

W5-M39 [what] `insert_at(2, x)` on a **singly** linked list of 5 nodes. How many references held by nodes that were **already** in the list change?

- **a)** none
- **b)** 2
- **c)** 3, the nodes after the gap
- **d)** 1, `prev.next`

W5-M40 [why] A singly linked list with `head` and `_tail` is reversed in place. What must happen to `_tail`?

- **a)** Nothing: the last node does not change
- **b)** It must refer to the old first node, which can be saved before the loop
- **c)** It must be found by a walk after the loop, an extra $O(n)$
- **d)** It must be set to `None`

Part B — Short answer and essay

W5-E1 [why] *(4 marks)* Compare a dynamic array and a singly linked list on four operations, with their costs. For each structure, give one situation where you would choose it, and say why.

W5-E2 [how] *(4 marks)* Explain `push_front` step by step with a diagram. Show what goes wrong if the two assignments are done in the other order.

W5-E3 [why] *(3 marks)* `LinkedList` supports `ll[i]`, just like an array. Explain why the cost is different, and why `for i in range(len(ll)): ll[i]` is a bad loop. What should be written instead?

W5-E4 [why] *(4 marks)* A tail reference makes `append` $O(1)$. What does it cost? Name the methods that must change, and explain why removing the last node is still $O(n)$.

W5-E5 [why] *(3 marks)* Explain what a doubly linked list and a sentinel node each fix, and what each costs.

W5-E6 [why] *(4 marks)* A music player keeps a playlist. The user presses *next* and *previous*, and can delete the song that is playing. Which structure would you choose: a dynamic array, a singly linked list or a doubly linked list? Give the cost of each of the three actions in your choice, and say why each of the other two is worse.

W5-E7 [how] *(4 marks)* Explain `insert_at(index, value)` step by step: the check on `index`, the case `index == 0`, the walk, and the splice. Why does the walk stop after `index - 1` steps? Give the best-case and worst-case cost, and show what goes wrong if the loop is written `for _ in range(index):`, both for an index in the middle and for `index == len(ll)`.

W5-E8 [why] *(3 marks)* A linked list and a dynamic array are both $O(n)$ to walk, yet measurement shows the array is faster and smaller. Explain both differences, with numbers for CPython. When is the linked list still the right choice?

W5-E9 [why] *(4 marks)* Explain how `reverse()` reverses a list in place with three references. State what `prev` and `current` mean at the start of each pass of the loop, why `following` must be saved first, and why the extra space is $O(1)$. Compare with building a reversed copy with `push_front`.

Part C — Trace the code

W5-T1 [how] What is printed?

```
head = None
for v in [1, 2, 3]:
    head = Node(v, head)
node = head
while node is not None:
    print(node.value, end=" ")
    node = node.next
```

W5-T2 [how] What is printed?

```
a = Node(1, Node(2, Node(3, Node(4))))
a.next = a.next.next
print(a.value, a.next.value, a.next.next.value, a.next.next.next)
```

W5-T3 [how] What does `mystery` return for chains of 5 nodes, 6 nodes, 1 node, and for `None`? What does it compute?

```
def mystery(head):
    count = 0
    node = head
    while node is not None and node.next is not None:
        node = node.next.next
        count += 1
    return count
```

W5-T4 [how] Trace `reverse()` on the list 1 → 2 → 3. Give `prev`, `node` and the links after each step.

W5-T5 [how] What is printed? What kind of structure has been made?

```
x = Node("a")
y = Node("b", x)
x.next = y
print(y.next.next.value, x.next.next is x)
```

Part D — List state

W5-S1 [how] Draw the list after each line, and give `len(ll)` and `ll.find(3)` at the end.

```
ll = LinkedList([1, 2, 3])
ll.push_front(0)
ll.insert_at(2, 9)
ll.pop_front()
ll.remove(2)
ll.append(5)
```

W5-S2 [why] A student writes `push_front` as

```
new = Node(value)
self.head = new
new.next = self.head
```

Draw the list after `ll = LinkedList([3, 7]); ll.push_front(5)`. What does `list(ll)` do?

W5-S3 [how] Draw the three references and all the links, step by step, while `remove(7)` runs on 3 → 7 → 1 → 9.

Part E — Complexity analysis

W5-K1 [how] Give Θ and justify:

```
ll = LinkedList()
for i in range(n):
    ll.append(i)            # no tail reference
```

How would you build the same list in $\Theta(n)$ without a tail reference?

W5-K2 [how] Give Θ and justify, for a linked list `ll` of n nodes:

```
count = 0
for value in ll:
    if ll.find(value) != -1:
        count += 1
```

W5-K3 [why] A queue must support "add at one end" and "remove from the other". With a singly linked list and **both** head and tail references, which end should be the front of the queue, and why? What are the costs?

Part F — Find and fix the bug

W5-B1 [how]

```
def find(self, value):
    node, index = self.head, 0
    while node.next is not None:
        if node.value == value:
            return index
        node, index = node.next, index + 1
    return -1
```

W5-B2 [how]

```
def remove(self, value):
    prev = self.head
    while prev is not None and prev.next is not None:
        if prev.next.value == value:
            prev.next = prev.next.next
            self._size -= 1
            return True
        prev = prev.next
    return False
```

W5-B3 [how]

```
def pop_front(self):
    value = self.head.value
    self.head = self.head.next
    return value
```

W5-B4 [why]

```
def reverse(self):
    prev, node = None, self.head
    while node is not None:
        node.next = prev
        prev = node
        node = node.next
    self.head = prev
```

Part G — Write the code

In `practice/week05.py`; check with `pytest tests/test_practice_week05.py -v`. Each function takes the first `Node` of a chain; walk and re-link — no copying into a Python list.

W5-C1 [how] `middle_value(head)` — the middle value in **one** pass.

W5-C2 [why] `has_cycle(head)` — does the chain loop back on itself? $O(1)$ extra space. Why must the fast reference eventually meet the slow one?

W5-C3 [how] `merge_sorted_chains(first, second)` — merge by re-linking, stable.

W5-C4 [how] `remove_duplicates_sorted(head)` — in place.

W5-C5 [how] `kth_from_end(head, k)` — k = 1 is the last node.